

KHAZANA

Implementation & Hardening Plan · Version 3

From “feature-rich” to “financially trustworthy.”

This document consolidates the existing Implementation Plan v2 with the additional correctness, reliability, data-lifecycle, portfolio, tax, and production-readiness issues identified during the review of the current Khazana project.

Important: this PDF is an updated implementation specification and acceptance checklist. It does not claim that the code is already fixed. Each item becomes “resolved” only after its acceptance criteria and tests pass in the project.

Objective	Resolution
Financial numbers disagree between screens	One DateRange + PeriodService + canonical accounting rules
Current month is mixed with historical reports	Dedicated This Month surfaces + explicit historical ranges
Budget behavior is ambiguous	Monthly default + explicit period + rollover policy
Alerts are unclear in a serverless app	Explicit condition types + last evaluated time + minimal lifecycle
Ledger / import / recurring paths may diverge	Single posting path + reconciliation tests
Backup exists but restore is missing	Round-trip restore with six validation gates
Receipts silently fail to persist	Attachment lifecycle with persistence tests
Transactions cannot be edited safely	Edit → reverse/repost semantics with audit-safe invariants
Market data can become stale	Price provenance + freshness + staleness UX
Tax and portfolio capabilities are incomplete	Tax/lot-data readiness plan with explicit scope and deferred integrations

Target product principle

Every number must be traceable to a source, a period, and a deterministic rule. Every mutation must preserve the ledger. Every backup must be restorable. Every serverless feature must accurately communicate what is and is not being checked while the app is closed.

1. Executive Summary

The original v2 plan correctly identified the largest architectural risk: multiple screens were independently computing “current period,” which could make Budget, Dashboard, Reports, Alerts, Recurring, Goals, and portfolio prices disagree. It also correctly moved the reconciliation diagnostic away from the Balance Sheet and introduced a shared DateRange/PeriodService. [\[filecite\[turn5file0\[L23-L43\]](#)

This v3 plan keeps those decisions and adds the remaining defects identified during the review: backup restore, receipt persistence, transaction editing, unreachable AMFI provider code, market-price freshness, tax/data readiness, and a dedicated financial-integrity test suite.

Definition of Done for this hardening cycle

- No primary financial screen exposes internal reconciliation diagnostics.
- Dashboard, Budget, Transactions, Calendar, Analytics, Reports and Alerts use one shared period model.
- Current-month income, expense, net and savings rate are available without opening a 3M/6M/12M historical report.
- Historical reports explicitly show their resolved date range.
- Budgets are monthly by default, with explicit rollover behavior and derived spending.
- Manual, recurring, imported and captured transactions converge on the same balanced ledger posting path.
- A user can edit a transaction without corrupting historical balances or double-counting.
- Receipt attachments persist, can be opened, and are removed deterministically.
- Backup export can be restored and round-tripped through a fresh install.
- Alerts communicate their type, condition, last evaluation time and serverless limitations.
- Portfolio prices expose source/date/freshness and never silently present stale data as current.
- Tax calculations are deterministic, versioned, clearly labelled as estimates, and covered by fixture tests.
- A financial-integrity test suite proves reconciliation across ledger, accounts, budgets, reports and portfolio lots.

2. Updated Priority Model

The v2 sequencing was P0 Correctness → P1 Core UX → P2 Alert Engine → P3 Intelligence. The v3 plan retains that ordering but inserts reliability work before intelligence because a financial app cannot be considered trustworthy if its backup, editing, attachments, or core data lifecycle is incomplete.

Tier	Focus	Exit condition
P0	Financial correctness & period model	All core numbers reconcile and use shared period rules
P1	Current-month & historical UX	Users can clearly distinguish “this month” from historical analysis
P2	Alert engine	Every alert has explicit semantics and honest evaluation state
P3	Data lifecycle & production reliability	Edit/import/recurring/backup/attachments/market data are trustworthy
P4	Intelligence	Forecasting, Safe-to-Spend, reviews, projections and optional AI sit on deterministic facts

Release gate

Do not promote P3/P4 features to the main navigation merely because the UI exists. A feature is considered shipped only when its domain invariants, failure paths, persistence behavior, and tests are complete.

3. P0 — Financial Correctness

3.1 Single time model

Create one immutable `DateRange` value and one `PeriodService`. The v2 plan explicitly limits the first version to `MONTH` and `CUSTOM`, with `currentMonth()`, `previousMonth()`, `nextMonth()`, and `custom(a,b)`.

□filecite□turn5file0□L55-L58□

Rule	Implementation
Current month	Local calendar month from first day 00:00 through last day 23:59:59.999, using the app's configured financial timezone.
Previous/next	Derived only by <code>PeriodService</code> ; no screen-local date arithmetic.
Custom	Explicit inclusive/exclusive boundary contract documented once and reused everywhere.
3M / 6M / 12M	Presets resolve to concrete <code>DateRange</code> values before querying data.
Future	Forecast windows are separate from historical/current periods and are never mixed into actuals.

3.2 Canonical money classification

- Income: postings that increase the user's economic resources and are classified as income by the domain.
- Expense: postings that reduce economic resources and are classified as spending.
- Transfer: movement between owned accounts; never counted as income or expense.
- Investment purchase: cash outflow and portfolio acquisition; must not be accidentally counted as consumer spending.
- Investment sale/dividend: classified according to the portfolio/accounting model, with realized gain separated from cash movement.
- Liability principal, interest and fees remain distinct where the existing domain model supports the distinction.

3.3 Derived values

Budget spent remains derived from transactions for the active period, as specified in v2; it is never stored as a mutable counter that can drift from the ledger. □filecite□turn5file0□L68-L70□

3.4 Balance Sheet diagnostic leak

The existing reconciliation difference must move entirely to Diagnostics. The Balance Sheet is allowed to show only financial statements and a concise balanced/unbalanced state; developer-level evidence belongs in Diagnostics. The v2 plan identifies this as the highest current trust risk. □filecite□turn5file0□L49-L54□

3.5 Financial Integrity Test Suite — new P0 requirement

Test family	Required invariant
Ledger	Every transaction is balanced; no orphan postings; debit/credit totals reconcile.
Accounts	Account balance equals opening balance plus all applicable postings.
Transfers	Transfers change account balances but do not inflate income or expense.
Budget	Spent = transaction sum for category + <code>DateRange</code> ; remaining = allocation + rollover – spent.
Reports	Income – expense = net for the same <code>DateRange</code> ; all report totals reconcile to ledger queries.
Balance Sheet	Assets – liabilities – equity reconciles within the configured tolerance.
Portfolio lots	Quantity, cost basis, realized P&L; and unrealized P&L; reconcile from lots and price snapshots.

Test family	Required invariant
Periods	The same DateRange produces identical totals wherever the same metric appears.
Import/recurring	Both use the same posting service as manual transactions.

4. P1 — Current-Month & Historical UX

4.1 Dedicated “This Month” experience

The v2 plan explicitly separates the daily financial home from historical Reports. Dashboard must show current-month income, expenses, savings rate, categories, budget status and upcoming items.

[filecite\turn5file0\L79-L86](#)

Surface	Default period	Required visible label
Dashboard	Current month	August 2026 · 1 Aug–31 Aug
Transactions	Current month	August 2026
Calendar	Current month	August 2026
Budget	Current month	August 2026 · Monthly · Rollover Off/On
Reports	Historical preset	Historical · 3M · Jun 1–Aug 31, 2026
Analytics	Selected historical/current period	Resolved DateRange
Alerts	Rule-defined period	Condition + last evaluated

4.2 Reports: 3M / 6M / 12M / Custom

Reports becomes explicitly historical. The selector should not merely say “3 months”; it should resolve to a date range such as “Jun 1–Aug 31, 2026.” The v2 plan explicitly requires this. [filecite\turn5file0\L84-L89](#)

Recommended report controls

- Historical: 3M · 6M · 12M · Custom.
- Current: This Month is a separate shortcut, not an accidental report window.
- Previous/next month controls where month navigation is meaningful.
- Export includes the resolved date range in the report metadata.
- Every KPI and chart tooltip carries the same resolved period.

4.3 Budget model

Behavior	Decision
Default period	Monthly
Weekly	Supported only where explicitly selected
Custom	Supported for advanced cases
Rollover	Off by default; opt-in per category
Spent	Derived from ledger transactions; never stored
Remaining	Derived; never stored
Progress zone	Derived from percentage; one shared rule across all screens
Alert threshold	Explicit percentage or explicit rupee amount

The design source already describes monthly envelopes, derived progress zones, and a default 90% alert threshold; v3 keeps those concepts but makes their period and persistence contracts explicit.

[filecite\turn4file2\L133-L152](#)

5. P2 — Alert Engine

5.1 Serverless reality

Khazana has no backend, push service or continuous worker. Therefore an alert is a local rule evaluated when the app opens, when the relevant screen is refreshed, or when a supported local scheduler executes while the client is available. The UI must never imply cloud monitoring.

5.2 Alert types

Type	Examples	Evaluation
Financial	Budget $\geq$ 90%; spending exceeds ₹X	Deterministic query against local ledger
Scheduled	Bill due in 3 days	Date comparison against local recurring/scheduled data
Market / portfolio	Price below ₹X; position weight above Y%	Uses stored price snapshot and freshness metadata

5.3 Alert record

Field	Purpose
id	Stable local identifier
type	FINANCIAL / SCHEDULED / MARKET
condition	Typed condition, not ambiguous free text
state	ARMED / TRIGGERED / DISMISSED
lastEvaluatedAt	When the rule was last checked
lastTriggeredAt	When it most recently became true
dataAsOf	Optional source-data timestamp, especially for market rules
message	Human-readable rendering generated from the typed condition

This preserves the v2 requirement for explicit condition language and lastEvaluatedAt, while adding dataAsOf for market-data honesty. The original plan correctly rejected a five-state lifecycle in favor of a minimal three-state model. [filecite:turn5file0L96-L110](#)

5.4 Examples

Bad	Good
Budget exceeded 300	Food spending is 90% of the August 2026 budget (₹7,200 of ₹8,000).
Stock alert	HDFCBANK fell below ₹1,500. Price data as of 9 Aug 2026, 6:00 PM.
Bill alert	HDFC Home Loan EMI is due in 3 days (12 Aug 2026).

6. P3 — Data Lifecycle & Production Reliability

6.1 Backup restore — new mandatory reliability feature

The project overview identified a critical asymmetry: backup export exists but restore does not. The existing PRD quality gate also states that a backup round-trip should export, uninstall, reinstall, restore and recover transactions and holdings. This becomes a mandatory v3 release gate.

Restore workflow

- Validate magic header FTOS.
- Validate supported schema version.
- Validate encrypted payload structure and nonce.
- Verify pre-encryption database SHA-256 according to the specified backup contract.
- Derive the key from the user's PIN and decrypt.
- Validate database integrity before replacing the active vault.
- Restore atomically to a new local database; never overwrite the current vault before validation succeeds.
- Open the restored vault and run the financial-integrity suite.

Acceptance test

Export → uninstall/delete local database → reinstall → restore → unlock → verify all transactions, accounts, holdings, lots, budgets, goals and settings. The existing PRD explicitly defines backup round-trip as a quality gate. [filecite:turn4file1L101-L110](#)

6.2 Receipt attachment persistence — new mandatory fix

Requirement	Acceptance criterion
Save	After attaching a receipt and saving a transaction, the attachment exists after app restart.
Open	Attachment can be opened from transaction detail.
Edit	Editing the transaction does not orphan the attachment.
Delete	Deleting the transaction removes or explicitly tombstones its attachment according to the storage contract.
Backup	Attachments are included in backup or explicitly excluded with a visible warning; preferred behavior is inclusion.
Failure	Storage errors are surfaced; no silent success state.

6.3 Transaction editing — new mandatory feature

A finance app needs correction workflows. Editing must not directly mutate a ledger in a way that breaks historical postings. The implementation should use a domain-level edit operation that validates the original transaction, reverses or replaces its postings atomically, writes the corrected postings, and re-runs invariants.

- Edit amount, date, merchant, category, note and account only through the transaction service.
- Prevent edits that would leave an unbalanced posting set.
- Preserve the original transaction identity/history according to the domain model.
- Recalculate budgets and reports from the corrected ledger, not cached totals.
- Provide a confirmation for high-impact changes such as amount/account/date.

6.4 Recurring and import paths

The v2 plan already requires recurring and import flows to use the same posting path as manual entry.
v3 adds idempotency and rollback tests so failures cannot leave partial postings.

7. P3 — Portfolio & Market-Data Integrity

7.1 Price freshness

Khazana's portfolio model intentionally stores dated price series rather than a single price column. That design should be surfaced in the product instead of hiding staleness.

Field	Required behavior
instrument	Stable symbol / ISIN identity
price	Decimal monetary value
priceAt	Timestamp/date of observation
source	Provider/import/manual source identifier
currency	Price currency
status	Fresh / Stale / Missing / Manual

UX rules

- Every market price shown in a portfolio metric can reveal its as-of date.
- Stale prices are visually distinct but not alarmist.
- Missing prices never silently become zero.
- Portfolio valuation with stale prices shows an explicit “valuation based on stale data” notice.
- Alerts based on stale prices must not pretend to be current market alerts.
- Latest-price selection remains deterministic and Decimal-safe; do not use lexicographic SQL MAX on Decimal stored as TEXT.

7.2 AMFI NAV provider: use, isolate, or remove

The project overview identified an AMFI NAV provider that is fully tested but has no callers. This is not a feature-completeness problem; it is an architecture hygiene problem.

Decision	Action
Use	Put provider behind a MarketDataProvider interface and add a real caller + integration tests.
Keep for later	Mark it explicitly as unreachable/deferred and prevent it from being mistaken for a production path.
Remove	Delete dead provider code and its tests if the product no longer intends to use it.

7.3 Portfolio invariants

- Buy quantity increases the lot quantity and cost basis by principal + applicable charges.
- Sell reduces matching lots and realizes gain/loss according to the configured lot-selection method.
- Charges are part of cost basis where specified by the product model.
- Dividends are cash income and portfolio events according to the canonical domain classification.
- Unrealized P&L; uses the selected price snapshot and clearly states its as-of date.
- XIRR fixtures include irregular cash flows, dividends, partial sells and charges.

8. P3 — Tax Center & India Tax Readiness

The current product already models a tax estimator and the design source describes India FY25 rules, equity STCG/LTCG, holding periods, exemptions and position-by-position estimates. [\[filecite\]turn4file9\[L1097-L1121\]](#) The hardening plan must make the estimator deterministic and versioned rather than treating UI output as authoritative.

Tax engine contract

Requirement	Rule
Tax rules	Versioned rule-set with effective date; never hard-code unexplained rates in UI.
Lot selection	Explicit FIFO or configured method; tests must match the chosen contract.
Holding period	Calculated from acquisition/trade date according to asset-class rule.
Charges	Included/excluded consistently according to the tax model and documented.
Exemption	Applied only where the applicable rule set permits it.
Unrealized estimate	Clearly labelled “if realised today” / estimate; never presented as a filing result.
Data provenance	Every estimate can identify the rule-set version and data date.

Deferred integrations — now explicitly tracked

- CAS import.
- AIS / 26AS import.
- Tax-harvesting suggestions.
- Filing-ready computation.
- External tax-account integration.

These remain deferred product features, not hidden omissions. They require a stable lot, transaction and tax-rule foundation first.

9. P4 — Intelligence Built on Deterministic Facts

The original v2 plan correctly defers intelligence until the financial model is stable and proposes Facts → Rules → Insights → optional AI explanation. [filecite\turn5file0\L113-L125]

Feature	Source facts	Output
Cash-flow forecast	Income, expenses, recurring rules, account balances	Projected cash position
Safe-to-Spend	Budget remaining, days remaining, recurring obligations	Daily/period spending allowance
Monthly review	Current vs previous month	What changed and why
What-if simulator	Debt payoff + goals + cash flow	Scenario projection
Net-worth projection	Assets, liabilities, goals, assumptions	Projection, never prediction
AI explanation	Deterministic facts and rules	Plain-language explanation only

Safe-to-Spend contract

Safe-to-Spend must never be a magic number. The screen should show the calculation inputs: available spendable cash, remaining budget, days remaining, committed recurring obligations and the period used. If an input is missing, the app should say “Not enough data” rather than fabricate a value.

10. Navigation & UI Consistency

The current web client has a large navigation surface. The v2 plan intentionally deferred a sidebar restructuring project. Keep that decision: correctness work should not be mixed with information-architecture refactoring. However, the following consistency rules are mandatory now.

- Dashboard is the daily home and always exposes current-month money status.
- Reports is historical analysis; its period is explicit.
- Budget is a period-based planning tool, monthly by default.
- Transactions is the authoritative user-facing timeline of posted/captured financial events.
- Diagnostics is the home for reconciliation, migration, provider and integrity problems.
- Alerts are local rule evaluations, not cloud notifications.
- Every screen that shows money has a visible period or “as of” timestamp where applicable.
- Every market-data value can reveal freshness.
- No diagnostic message should leak into a primary financial statement.

Theme consistency

The existing design direction uses a near-black/green dark theme and a white/very-light green light theme. The light theme should darken the interaction green for contrast while preserving the same brand hue. The existing design specification uses #20C98A in dark mode and #087A56 in light mode, with #F5F7F5 / #FFFFFF surfaces.

11. Test & Verification Matrix

Area	Unit	Integration	E2E / acceptance
PeriodService	DateRange boundaries	All consumers	Dashboard/Budget/Reports agree
Ledger	Balanced postings	Posting service	Create/edit/import/recurring reconcile
Budget	Derived spent	Period + ledger	Monthly/rollover scenarios
Reports	Metric formulas	Ledger queries	3M/6M/12M/Custom UI
Alerts	Condition evaluation	Data freshness	Triggered/dismissed + timestamp
Backup	Header/hash/decrypt	Restore DB	Round-trip after reinstall
Attachments	Persistence/delete	Transaction linkage	Restart + backup restore
Portfolio	Lot math/XIRR	Price snapshots	Stale/missing price UX
Tax	Rule fixtures	Lot-to-tax pipeline	Position-by-position estimate
SMS capture	Parser fixtures	Draft creation	Raw message discarded immediately

Mandatory regression scenarios

- Create salary → expense → transfer → investment purchase; verify income, expense, net, account balances and portfolio all agree.
- Import the same statement twice; second import creates no duplicate financial events.
- Create recurring transaction; verify it posts exactly once and is balanced.
- Edit an existing transaction; verify budget, reports, ledger and account balance all change consistently.
- Attach receipt → restart → reopen transaction → backup → restore → verify attachment.
- Create budget with rollover off/on and test month boundary.
- Evaluate budget alert with 89%, 90%, 100% and rupee-threshold conditions.
- Use a stale market price; verify portfolio and alert copy expose the as-of date.
- Export and restore backup after a clean reinstall.
- Corrupt backup or enter wrong PIN; verify the error path does not reveal which condition occurred.

12. Production-Readiness Gate

Khazana should not be called production-ready merely because the screens are complete. Use this checklist as the final gate.

Gate	Pass condition
Accounting	All ledger and balance-sheet invariants pass.
Periods	Every period-bound number has one authoritative DateRange.
Current month	Income/expense/net/savings are available as a dedicated current-month view.
Historical reports	3M/6M/12M/Custom ranges are explicit and reproducible.
Budgets	Monthly default, rollover explicit, spending derived.
Transactions	Create/edit/delete flows preserve ledger integrity.
Import	Idempotent and ledger-backed.
Recurring	Ledger-backed and idempotent.
Attachments	Persist and survive restart/restore.
Backup	Full export/restore round-trip succeeds.
Alerts	No false real-time implication; conditions and evaluation time visible.
Market data	Freshness/source/as-of are visible; stale values are not silently treated as current.
Tax	Rule-set versioning and fixture coverage exist.
Diagnostics	All internal inconsistencies are discoverable without polluting user statements.
Privacy	Core workflows operate without a backend and no user financial data is sent to a server.

Release recommendation

Ship only after P0 + P1 + the reliability gates in P3 are green. P2 alerts can then be enabled. P4 intelligence should follow only after deterministic financial facts are proven by the integrity suite.

13. Final Implementation Order

Order	Work item	Result
1	Move reconciliation diff to Diagnostics	Clean Balance Sheet
2	Implement DateRange + PeriodService	One time model
3	Canonical income/expense/transfer rules	One accounting vocabulary
4	Wire Dashboard/Budget/Transactions/Calendar/Reports	No period drift
5	Dedicated This Month view	Daily financial home
6	Historical 3M/6M/12M/Custom ranges	Clear analysis windows
7	Budget monthly + rollover + derived values	Predictable planning
8	Ledger convergence for manual/import/recurring/edit	Single posting path
9	Financial-integrity test suite	Provable correctness
10	Alert engine + lastEvaluatedAt	Honest local alerts
11	Backup restore	Recoverable vault
12	Receipt persistence	Complete transaction documents
13	Transaction editing	Correctable financial records
14	Portfolio price freshness + provider hygiene	Trustworthy valuation
15	Tax rule versioning + fixture tests	Deterministic tax estimates
16	Cash-flow forecast + Safe-to-Spend	First intelligence layer
17	Monthly review + what-if + projections	Decision support
18	Optional AI explanation layer	Natural-language explanation of proven facts

Final product rule

Khazana's differentiator is not the number of screens. It is that a user can trust the number on every screen, understand its period, trace it to local financial data, correct it safely, and recover the vault if the device or application state is lost.

Source basis

Primary source: Khazana · Implementation Plan v2. Additional requirements in this v3 document come from the project overview and the defects explicitly identified during the review: missing restore, receipt persistence, transaction edit, unreachable AMFI provider, market-data freshness, tax completeness, and the need for a financial-integrity regression suite.